

## Cost function formula

Cost function will tell us how well the model is doing so that we can try to get it to do better.

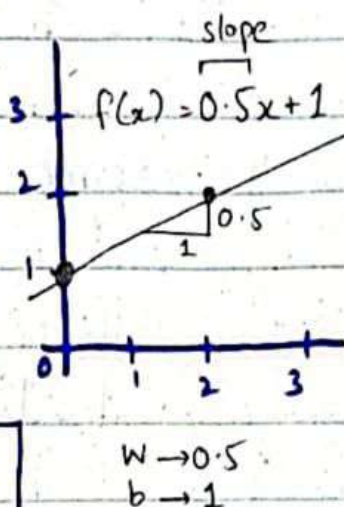
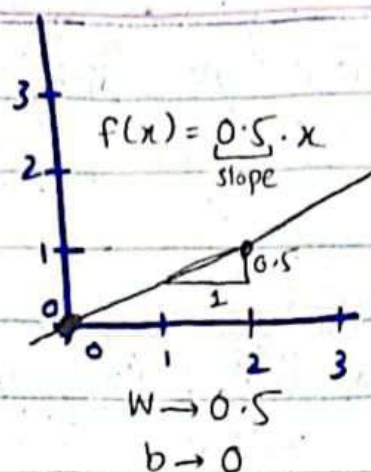
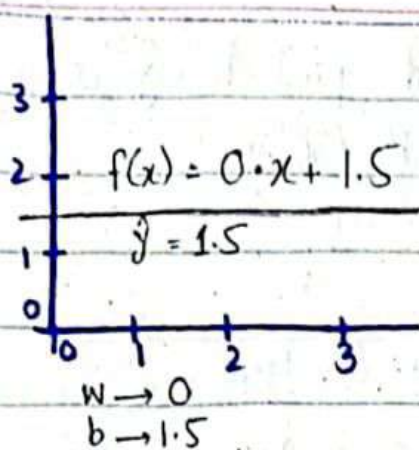
| features                  | targets            | Model:                |
|---------------------------|--------------------|-----------------------|
| Size in feet <sup>2</sup> | price \$1000's (y) |                       |
| 2104                      | 460                | $f_{w,b}(x) = wx + b$ |
| 1416                      | 232                |                       |
| 1534                      | 315                | $w, b$ : parameters   |
| 852                       | 178                | coefficients          |
| ...                       | ...                | weights               |

### Parameters:

In ml, parameters of the model are the variables you can adjust during training in order to improve the model.

### What do $w$ & $b$ do?

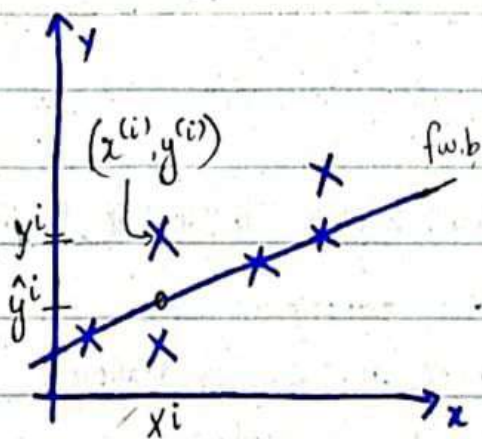
depending on the values you have chosen for  $w$  &  $b$  you get a different value for  $f(x)$ .



Slope: as for every step taken for right it goes 0.5 up

$\hat{y}^{(i)} = f_{w,b}(x^{(i)})$   
 $f_{w,b}(x^{(i)}) = wx^{(i)} + b \rightarrow$  for our model LR.

how much rise (up/d) you get for how much run (right) you went.



With linear regression, what you wanna do is choose values for params  $w$  &  $b$  so that the straight line we get from function  $x$  somehow fits the data well.

- fit well means the line defined by  $f(x)$  is roughly passing through or somewhere near to the training examples.



How we find  $w, b$ : so that prediction  $\hat{y}^{(i)}$  is close to true target  $y^{(i)}$  for all  $(x^{(i)}, y^{(i)})$ .  
or mostly

For that we use the "Cost function":

Takes  $\hat{y}$  (prediction) & compares that to  $y$  (target) by:

$$\hat{y} - y \rightarrow \text{error}$$

as we measure how far the prediction is from the target

Next compute their square:

$$(\hat{y} - y)^2$$

We'll also compute from diff training examples  $i$  in training set.

$$(\hat{y}^{(i)} - y^{(i)})^2$$

Finally, we wanna measure error across the entire training set.

$$\sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 \quad \text{where } m = \text{no. of training example}$$

as  $m$  grows larger, your cost function will calculate a greater number (summing over more examples).

To build cost function that doesn't automatically get bigger as  $m$  grows larger, we will compute "avg. squared error".

$$\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

One last thing, by convention the c.f. that ml people use actually divides by  $(2xm)$ . The extra division by 2 is just meant to make some of our later calculations look neater.

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

cost function  
also called  
"squared error"  
cost function

We will use different cost func for diff applications in ml, but this - sq. error c.f. is by far the most common function for linear regression & for that matter for all regression problem where it seems to give good results for many apps.



we can also write it as:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\cancel{f(w, b)} f_{w, b}(x^{(i)}) - y^{(i)})^2$$

$$\text{as } \hat{y}^{(i)} = f_{w, b}(x^{(i)})$$

Eventually we are gonna find values of  $w$  &  $b$  that makes cost function small but first we will get intuition of what  $J$  of  $w, b$  is actually computing.

---